

RECIPE SHELTER

Site Web de Recettes de Cuisine Collaboratif

DOCUMENTATION TECHNIQUE — BLOC 3 : FRAMEWORK ANGULAR

Justification du choix · Architecture · Routing · État · SSR · Tests

Élément	Détail
Framework	Angular 21
Langage	TypeScript 5.9 (strict mode)
Gestion d'état	Angular Signals (computed, signal, effect)
Routing	Angular Router — lazy loading par domaine
Formulaires	Reactive Forms (FormGroup, FormControl, validators)
HTTP	HttpClient + intercepteur fonctionnel (authInterceptor)
SSR	Angular SSR avec Express (renderMode par route)
Tests	Vitest — suite *.spec.ts collocalisée
Qualité	ESLint angular-eslint + Husky pre-commit

1. Justification du choix d'Angular

1.1 Pourquoi Angular ?

Angular a été choisi comme framework front-end pour les raisons suivantes :

- Framework full-featured : Angular fournit nativement le routeur, les formulaires réactifs, le client HTTP, l'injection de dépendances, le SSR et les outils de test. Aucune bibliothèque tierce n'est nécessaire pour les fonctionnalités de base.
- TypeScript natif : Angular est conçu autour de TypeScript, ce qui garantit la sécurité des types sur l'ensemble du front-end, en cohérence avec le back-end également en TypeScript.
- Architecture structurée : la séparation en composants, services, guards et intercepteurs impose une organisation claire du code, adaptée à un projet de cette envergure (17 pages, 3 niveaux de droits).
- Composants standalone : l'architecture moderne d'Angular sans NgModule simplifie les imports et réduit la quantité de code de configuration.
- Signals : la gestion réactive de l'état par Signals offre une alternative légère à RxJS pour les états locaux, avec une syntaxe claire et des performances optimisées.
- SSR intégré : Angular SSR avec Express permet le rendu côté serveur sans configuration externe complexe, améliorant le SEO et les performances de chargement initial.
- Proximité avec l'écosystème C#.NET : venant d'un background .NET, Angular est un choix naturel. Son architecture orientée services, son injection de dépendances et sa structure en modules rappellent les patterns ASP.NET MVC et les applications .NET en général. Angular est d'ailleurs fréquemment utilisé comme front-end dans les stacks .NET en entreprise, ce qui facilite la montée en compétences et l'intégration dans des équipes existantes.

1.2 Comparaison avec les alternatives

Critère	Angular 21 (choisi)	React / Vue
Routing	Intégré (Angular Router)	Bibliothèque externe (React Router, Vue Router)
Formulaires	Reactive Forms natifs	Bibliothèques tierces (React Hook Form, Formik...)
HTTP	HttpClient + intercepteurs natifs	fetch ou axios (tierces)
TypeScript	Natif, strict mode	Optionnel (React), supporté (Vue 3)
SSR	Angular SSR intégré	Next.js (React), Nuxt (Vue) — projets séparés
Structure	Convention forte, architecture imposée	Liberté totale (peut mener à l'incohérence)
État	Signals natifs	useState, Zustand, Pinia (tierces)

2. Architecture de l'application

2.1 Structure des dossiers

L'application est organisée en quatre zones distinctes, chacune avec une responsabilité claire :

Dossier	Rôle
core/	Services applicatifs globaux, guards, intercepteurs, modèles TypeScript et utilitaires. Injecté dans toute l'application.
layouts/	Composants structurels : Layout (conteneur principal), Header (navigation), Footer. Présents sur toutes les pages.
pages/	Domaines fonctionnels : auth, recipes, admin, profile, favorites, search, contact, legal, users, about. Chaque domaine a ses propres routes et composants.
shared/	Composants réutilisables inter-domaines : RecipeCard, CategoryIcon, validators communs, styles partagés.

2.2 Composants standalone

Tous les composants de l'application utilisent l'architecture standalone d'Angular (pas de NgModule). Chaque composant déclare ses propres imports directement :

```
@Component({
  standalone: true,
  selector: 'rs-recipe-card',
  imports: [CommonModule, RouterLink, DatePipe],
  templateUrl: './recipe-card.html',
  styleUrls: ['./recipe-card.css']
})
```

Le préfixe rs- est appliqué systématiquement à tous les sélecteurs de composants et directives, conformément à la convention documentée dans `RecipeShelterNamingConvention.md`.

2.3 Injection de dépendances

Les services utilisent `providedIn: 'root'` pour être des singletons au niveau de l'application, injectés via la fonction `inject()` d'Angular :

```
@Injectable({ providedIn: 'root' })
export class SessionService {
  private readonly _user = signal<SessionUser | null>(null);
  // ...
}
```

Cette approche garantit une instance unique par service dans toute l'application, sans déclaration dans un module.

3. Routing — Navigation et lazy loading

3.1 Architecture des routes

Le routing est organisé en deux niveaux : un fichier `app.routes.ts` racine qui définit le layout principal et charge les sous-routes par domaine fonctionnel, et des fichiers `*.routes.ts` par domaine (`auth.routes.ts`, `recipes.routes.ts`, `admin.routes.ts`...).

Tous les composants de page sont chargés en lazy loading via `loadComponent()` ou `loadChildren()`. Cela signifie que le code JavaScript d'une page n'est téléchargé que lorsque l'utilisateur y accède, réduisant le bundle initial.

3.2 Guards de navigation

Guard	Condition	Routes protégées
<code>authGuard</code>	Vérifie la session via GET <code>/auth/me</code> . Si non connecté : redirige vers <code>/sign-in</code> avec le paramètre <code>redirectTo</code> .	<code>profile</code> , <code>me/recipes/*</code> , <code>me/favorites</code>
<code>guestGuard</code>	Vérifie que l'utilisateur n'est PAS connecté. Si connecté : redirige vers <code>/</code> .	<code>sign-in</code> , <code>sign-up</code> , <code>forgot-password</code> , <code>reset-password</code>
<code>adminGuard</code>	Vérifie que <code>roleId === ROLE_IDS.ADMIN</code> . Si non admin : redirige vers <code>/</code> .	<code>admin/dashboard</code> , <code>admin/recipes/*</code> , <code>admin/comments/*</code> , <code>admin/users/*</code>

Le `authGuard` gère également le cas SSR : sur le serveur (`isPlatformServer`), il retourne `true` directement sans appel HTTP, car la session ne peut pas être vérifiée côté serveur sans cookie.

3.3 Modes de rendu par route (SSR)

Chaque route est configurée avec un mode de rendu défini dans `app.routes.server.ts` :

Mode de rendu	Routes concernées	Justification
<code>RenderMode.Prerender</code>	Toutes les routes non listées (<code>/**</code>)	Pages statiques (accueil, mentions légales, à propos) : rendues à la compilation pour un chargement optimal.
<code>RenderMode.Server</code>	<code>recipes/:slug</code> , <code>users/:username</code>	Pages dont le contenu dépend de paramètres d'URL : rendues côté serveur à chaque requête pour le SEO.
<code>RenderMode.Client</code>	Toutes les pages authentifiées (<code>me/*</code> , <code>admin/*</code> , <code>profile</code> ...)	Pages nécessitant une session : rendues côté client uniquement, car elles dépendent du cookie de session.

4. Gestion de l'état avec Angular Signals

4.1 Choix de Signals vs RxJS

Angular Signals (introduits en Angular 17) sont utilisés pour la gestion de l'état réactif local et global. Par rapport à RxJS Observable, les Signals offrent une syntaxe plus simple pour les états synchrones, une mise à jour automatique des composants qui dépendent d'un signal, et une meilleure lisibilité du code.

RxJS est conservé pour les opérations asynchrones (appels HTTP via HttpClient) et les transformations de flux complexes (pipe, map, catchError).

4.2 SessionService — état global de la session

Le SessionService est le gestionnaire d'état central de l'application. Il utilise trois types de Signals :

Type de Signal	Utilisation dans SessionService
signal<T>	_user : stocke l'utilisateur connecté. Privé, modifiable uniquement par les méthodes du service.
computed(()=>...)	user, isAuthenticated, isAdmin : valeurs dérivées calculées automatiquement à partir de _user. En lecture seule depuis l'extérieur.
Méthodes set/clear	setUser(), setAuthUser(), updateUser(), clear() : seuls points d'entrée pour modifier l'état.

Dans les composants, les Signals sont utilisés directement dans les templates Angular sans pipe async :

```
@if (session.isAuthenticated()) {  
  <span>Bonjour {{ session.user()?.username }}</span>  
}
```

4.3 État local des composants

Les composants utilisent des Signals locaux pour gérer leurs propres états (chargement, erreurs, données) :

```
readonly isLoading = signal(false);  
readonly errorMessage = signal<string | null>(null);  
readonly recipe = signal<PublicRecipeDetail | null>(null);
```

Ce pattern remplace l'usage de propriétés de classe ordinaires et garantit que les mises à jour déclenchent automatiquement le re-rendu du composant.

5. Interactivité et communication avec l'API

5.1 Intercepteur HTTP — authInterceptor

L'authInterceptor est une fonction interceptrice fonctionnelle (HttpInterceptorFn) qui s'applique à toutes les requêtes HTTP de l'application. Son rôle est double :

- Ajouter credentials: 'include' sur toutes les requêtes vers l'API, pour que le cookie rs_session soit transmis automatiquement par le navigateur.
- Intercepter les réponses 401 (non autorisé) et effacer la session locale si l'erreur n'est pas marquée PRESERVE_SESSION_ON_UNAUTHORIZED.

L'intercepteur est enregistré dans app.config.ts via `provideHttpClient(withInterceptors([authInterceptor]))`, sans NgModule.

5.2 Services HTTP par domaine

Chaque domaine fonctionnel a son service Angular dédié pour les appels API :

Service	Routes API couvertes	Particularités
AuthService	POST /auth/login, /register, /logout, GET /auth/me, POST /forgot-password, /reset-password, /validate-email	Gère tous les flux d'authentification
RecipesService	GET /recipes, /recipes/search, /recipes/:slug, POST/PUT /me/recipes/:id, POST .../submit	Construction dynamique des queryParams pour la recherche avancée
CommentsService	POST /recipes/:id/comments, PUT/DELETE .../comments/:id	Gestion des commentaires et réponses imbriquées
FavoritesService	GET /me/favorites, POST/DELETE /me/favorites/:id	Mise à jour optimiste de l'état favori
AdminRecipesService	GET/POST /admin/recipes, /admin/recipes/:id/approve, /reject	Routes réservées aux administrateurs
AdminCommentsService	GET /admin/comments/moderated, PATCH .../moderate	Modération des commentaires
AdminUsersService	GET /admin/users, PATCH /admin/users/:id/ban, /unban	Gestion des bannissements

5.3 Interactions sans rechargement de page

Les interactions utilisateur clés sont traitées sans rechargement via HttpClient (AJAX) :

- Ajout/suppression de favoris : le bouton cœur met à jour l'état local immédiatement (mise à jour optimiste), puis envoie la requête API. En cas d'erreur, l'état est rétabli.
- Soumission de commentaire : le formulaire envoie la requête POST et le nouveau commentaire est ajouté à la liste sans rechargement de la page.
- Soumission de recette : le formulaire multi-étape envoie les données et redirige vers la liste des recettes après confirmation.
- Modération admin : approbation, rejet ou modération depuis l'interface admin sans navigation.

6. Server-Side Rendering (SSR)

6.1 Fonctionnement

Angular SSR est configuré avec Express comme serveur Node.js. Au premier chargement, la page est rendue côté serveur (HTML complet envoyé au navigateur), puis Angular prend le relais côté client (hydratation). Les chargements suivants sont gérés côté client comme une SPA classique.

6.2 Bénéfices

- SEO : les moteurs de recherche reçoivent un HTML complet avec le contenu des recettes, leur titre, leur description et leurs métadonnées — sans devoir exécuter JavaScript.
- Performance perçue : le First Contentful Paint (FCP) est amélioré car le navigateur affiche immédiatement le HTML rendu, sans attendre le chargement du bundle JavaScript.
- Partage sur les réseaux sociaux : les previews (Open Graph) fonctionnent correctement car les métadonnées sont dans le HTML initial.

6.3 Configuration de l'hydratation

L'hydratation est activée avec l'option `withEventReplay()` qui rejoue les événements utilisateur survenus pendant le chargement, évitant les interactions perdues :

```
provideClientHydration(withEventReplay())
```

Le scroll est restauré en haut de page à chaque navigation avec `withInMemoryScrolling({ scrollPositionRestoration: 'top' })`.

7. Tests unitaires

7.1 Stratégie

Les tests unitaires couvrent les guards, intercepteurs, services et utilitaires. Chaque fichier source a son fichier de test collocalisé (*.spec.ts). Le framework de test est Vitest, configuré pour simuler l'environnement navigateur via jsdom.

7.2 Périmètre des tests

Fichier testé	Type	Ce qui est vérifié
auth.guard.spec.ts	Guard	Redirection vers /sign-in si session invalide, passage si session valide, comportement SSR
guest.guard.spec.ts	Guard	Redirection vers / si l'utilisateur est déjà connecté
admin.guard.spec.ts	Guard	Redirection si roleId !== ADMIN
auth.interceptor.spec.ts	Intercepteur	Injection de credentials: include, effacement session sur 401
auth.service.spec.ts	Service	Méthodes login, logout, register, me, forgotPassword, validateEmail
recipes.service.spec.ts	Service	getPublished, searchPublished (construction des queryParams), getBySlug
comments.service.spec.ts	Service	Création, mise à jour et suppression de commentaires
favorites.service.spec.ts	Service	Ajout et suppression de favoris
session.service.spec.ts	Service	Signals isAuthenticated, isAdmin, user — setUser, clear
auth-errors.spec.ts	Utilitaire	Mapping des codes d'erreur API (AUTH_INVALID_CREDENTIALS...) vers messages lisibles
recipe-status.spec.ts	Utilitaire	Libellés des statuts de recette (draft, pending, published, rejected, archived)
header.spec.ts	Composant	Rendu, liens de navigation, menu hamburger
footer.spec.ts	Composant	Rendu et liens du pied de page

7.3 Commande

```
npm run test
```

Les tests sont exécutés via ng test, qui utilise Vitest en environnement jsdom pour simuler le DOM du navigateur.

7.3 Couverture de code (v8)

La couverture a été mesurée avec le provider `@vitest/coverage-v8` via la commande `ng test --coverage`. Les 64 fichiers de tests ont été exécutés et les 153 tests sont tous passés avec succès.

Métrique	Statements	Branches	Functions	Lines
Global (All files)	73.92 %	73.01 %	67.52 %	75.82 %

Points forts

Plusieurs modules atteignent une couverture de 100 % : les validators (`password-match`, `password-strength`), les modèles, les pages légales, les layouts simples, et plusieurs composants de formulaire (`recipe-equipments-form`, `recipe-ingredients-form`). L'intercepteur d'authentification est couvert à 91 % et le session service à 90 %.

Axes d'amélioration

Les services Angular (`auth`, `recipes`, `favorites`, `comments`...) font majoritairement des appels HTTP. Ils sont testés en isolation avec mock du `HttpClient`, mais les branches liées aux cas d'erreur réseau restent non couvertes. Couvrir ces scénarios d'échec permettrait d'atteindre un taux global supérieur à 90 %.

8. Bilan des choix techniques Angular

Choix	Justification
Composants standalone	Simplifie la structure : plus de <code>NgModule</code> à maintenir. Chaque composant est autonome et ses dépendances sont déclarées explicitement.
Signals pour l'état	Syntaxe plus lisible que <code>RxJS</code> pour les états synchrones. Mise à jour automatique du DOM. <code>SessionService</code> lisible et prévisible.
Lazy loading systématique	Réduit le bundle initial. Chaque domaine fonctionnel est chargé à la demande.
Guards fonctionnels	Syntaxe moderne Angular 15+ (<code>CanActivateFn</code>). Plus légers que les classes implémentant <code>CanActivate</code> .
<code>authInterceptor</code> fonctionnel	Pas de classe : la fonction interceptrice s'enregistre directement dans <code>provideHttpClient()</code> . Moins de boilerplate.
SSR avec <code>renderMode</code> par route	Stratégie hybride : prerender pour les pages statiques, SSR pour les pages dynamiques SEO-critiques, client-only pour les pages authentifiées.
<code>withEventReplay()</code>	Évite les interactions perdues pendant l'hydratation — améliore l'expérience utilisateur sur les connexions lentes.
Préfixe <code>rs-</code> sur tous les sélecteurs	Identifie clairement les composants du projet, évite les conflits avec Bootstrap ou d'autres bibliothèques.